Architectures and Platforms for Artificial Intelligence

Programming Assignment 2024/2025

Alessio Pittiglio

1 Introduction

The assignment consists of implementing the inference for a multi-layer Neural Network (NN) with a particular structure. For simplicity, we first consider the case of only two layers. Let N be the number of nodes in the input layer, each containing x_i, with i ranging from 0 to $N-1$. The output layer has $N-(R-1)$ nodes, each denoted by y_j, with j ranging from 0 to $N-R$. The relationship between an output node and an input node can be expressed as follows:

$$y_i = f\left(\sum_{r=0}^{R-1} x_{i+r} \times W_{i,r} + b\right)$$

where $f(x)$ represents a generic activation function, which in our case has been chosen as the sigmoid function. In this formulation, the bias b is the same for all nodes across the layers. It is important to note that the output value y_i depends only on R nodes: we are therefore dealing with a sparsely connected network. This notation can be generalized to the case with K layers. R is assumed to be small.

The assignment requires two implementations: one using OpenMP and one using CUDA. The goal is to compare the performance of the two implementations.

2 Implementation Strategy

2.1 Common Design Choices

There are two possible interpretations of the weight matrix. The first interpretation suggests that each neuron in the output layer is computed using a set of R weights, which are different for each layer. The second interpretation, which is more faithful to the mathematical formulation, assumes that the weights are the same for every layer. This implies a single set of weights is reused to compute the transition from any layer t to layer $t+1$. The number of weights required is therefore R multiplied by the number of neurons in the first layer, $(N-R+1)$.

In the remainder of this work, we adopt the second interpretation. From an implementation standpoint, the weight matrix can represented as a 1D vector in row-major order, where all R weights for a given neuron are stored consecutively.

Another implementation decision concerns memory management. The chosen approach is a Ping-Pong scheme: two memory buffers, A and B, are used, both of size N. For each layer t, data is read from A and written to B. Afterwards, the roles of the buffers are swapped: we read from B and overwrite the new values into A. From an implementation point of view, this is achieved by swapping indices without changing the pointers. Although the second buffer is larger than necessary, this approach avoids costly memory reallocations and simplifies the logic.

2.2 OpenMP Implementation

While the layers must be evaluated sequentially, the computation of each neuron within the same layer depends only on the values of the previous layer and is independent of the computation of other neurons in the same layer. Therefore, the problem is embarrassingly parallel. This means it can be easily parallelized using OpenMP directives. The `schedule(static)` directive distributes iterations statically (the first block of iterations to thread 0, the second to thread 1, and so on). This is the optimal choice because the amount of work required to compute each output neuron is the same for all threads. Additionally, we explicitly declare that the pointers to the input layer, output layer, and weight vector are shared among all threads, which is necessary since all threads must read from and write to the same memory buffers.

Another optimization that has been applied is loop unrolling, which we have empirically observed to reduce execution times. Indeed, loop unrolling eliminates branches and increases instruction-level parallelism (ILP).

A further minor optimization that improved performance is initializing the sum from the bias instead of zero, saving one addition per output neuron, for a total of approximately $N \times K$ fewer additions.

2.3 CUDA Implementation

On the GPU, the bottleneck is almost always memory bandwidth. Therefore, the goal becomes minimizing transfers from global memory. Two different kernels were implemented: one using global memory and one using shared memory. NVCC already performs loop unrolling by default.

Global memory. The first kernel is a direct translation of the parallel logic. Each thread computes its global index, performs R reads from the input buffer and R reads from the weight matrix W in global memory, computes the sum, and writes the result back to global memory.

Shared memory. To mitigate the cost of global memory accesses, a second kernel was implemented that exploits on-chip shared memory. The strategy is to load a portion of the input x that is reused by threads within the same block. This is possible because the windows required for each output neuron overlap: thread i and thread $i + 1$ share $R - 1$ input values. During the tiling phase, each block collaboratively loads a portion of the input x into a shared memory array, synchronizes and then performs the computation. Only the first threads in each block load the necessary halo elements. This approach reduces accesses to global memory: shared memory transforms five reads from global memory into one read from global memory and five reads from shared memory. However, accesses to the weight matrix W remain problematic: thread i reads from `W[i * R]`, while thread $i+1$ reads from `W[(i+1) * R]`. This results in non-coalesced memory accesses, which are inefficient on GPUs.

3 Performance Evaluation

The performance evaluation was conducted on a local machine equipped with an Intel Core i5-12600KF CPU with 10 cores and 16 threads (6 Performance cores and 4 Efficient cores) and a 20 MB L3 cache. The GPU used was an NVIDIA RTX 3090. The operating system was Ubuntu 22.04 LTS. Compilation was performed with GCC 11.4.0 the flags `-std=c99 -Wall -Wpedantic -fopenmp`, and NVCC version

12.4.131. To calculate speedup and efficiency, wall-clock time was measured, excluding input and output operations. For this purpose, the `hpc` library provided by the professor was used. To obtain accurate performance results, each experiment was repeated five times, and the average execution time was taken. This minimizes the impact of system noise and interference from other processes.

3.1 OpenMP Scalability Analysis

Regarding the performance analysis of the OpenMP implementation, a scalability study was carried out, considering both strong scaling and weak scaling.

Strong scaling. Strong scaling measures how the execution time decreases as the number of processors (p) increases for a fixed problem size (N). The ideal speedup is linear: doubling the number of processors halves the execution time.

For the strong scaling experiment, we selected a problem size of $N = 10^6$ and $K = 1500$, which resulted in a serial execution time of 9.8 seconds. It is important to analyze the memory footprint of this setup. We assume $R = 5$. The two layer buffers require $2 \times 10^6 \times 4$ bytes ≈ 8 MB, while the weight matrix requires $10^6 \times 5 \times 4$ bytes $\approx$ 20 MB, for a total of approximately 28 MB. Since the working set exceeds the L3 cache size, we expect the execution to be limited by memory bandwidth (memory-bound), meaning that adding more computational cores does not lead to a linear improvement in performance, as all cores contend for access to the memory bus (an effect known as *memory wall*).

The speedup is defined as:

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \approx \frac{T_{\text{parallel}}(1)}{T_{\text{parallel}}(p)}$$

In our case, the number of threads was varied from 1 to 16. Figure 1a shows the speedup relative to single-threaded execution. The implementation scales well up to the number of Performance cores (5.45×), after which the performance gains from hyper-threading diminish.

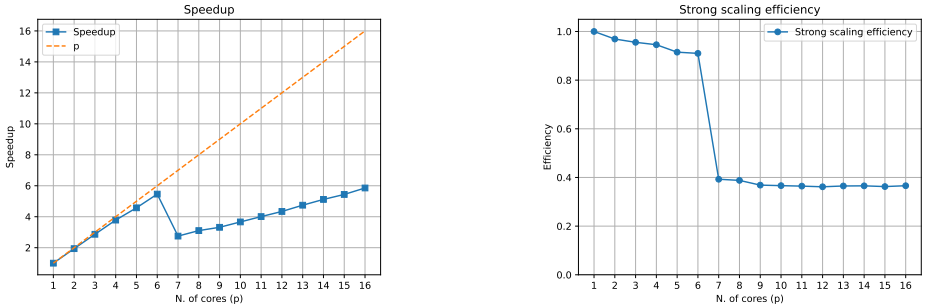
The parallel efficiency is defined as:

$$E(p) = \frac{S(p)}{p} = \frac{T_{\text{parallel}}(1)}{p \times T_{\text{parallel}}(p)}$$

where $T_{\text{parallel}}(p)$ is the execution time of the parallel program with p processors/cores. Figure 1b shows the efficiency. It remains close to 0.9 as long as only the Performance cores are used, but once hyper-threading comes into play, it drops to around 0.4, where it then stabilizes.

Weak scaling. Regarding weak scaling, the first step is to define the workload. In our implementation, the work is distributed by the `#pragma omp parallel for` directive inside `forward_pass`. The amount of work is directly proportional to $N - t \cdot (R - 1)$, which is itself almost directly proportional to N. The simplest rule is: if the number of threads is doubled, the problem size N is also doubled, keeping the workload per thread constant. The parameter K defines the sequential portion of the program. For a pure weak scaling analysis, it is better to keep K constant and scale only N.

The weak scaling performance can be expressed as:



(a) Speedup (b) Efficiency

Figure 1: Strong scaling results

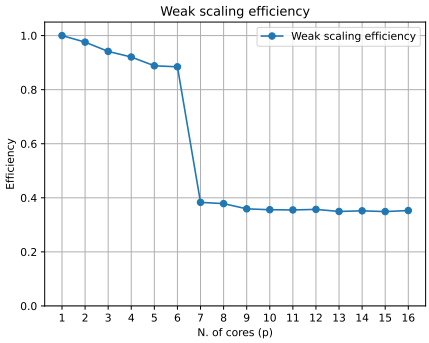


Figure 2: Weak scaling results ($N = 250{,}000$, $K = 1500$)

$$W(p) = \frac{T_1}{T_p}$$

where T_1 is the time required to complete one work unit with a single processor, and T_p is time required to complete p work units with p processors.

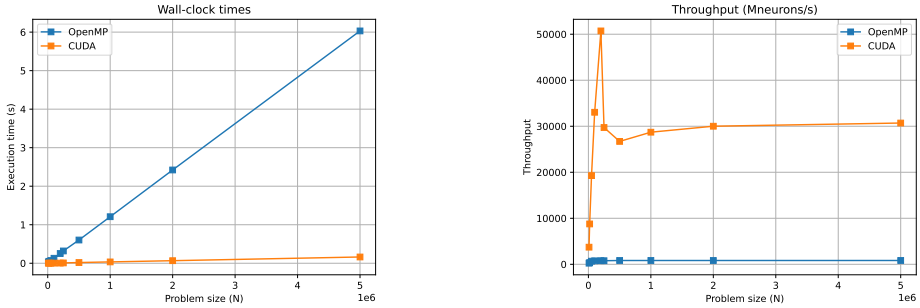
Figure 2 reports the parallel efficiency for the weak scaling experiment. The efficiency starts at 1 for a single thread and remains around 0.9 when only the Performance cores are used. When additional threads are used, the efficiency drops slightly below 0.4, where it then stabilizes.

3.2 CUDA Performance Analysis

Performance analysis of the CUDA implementation is typically carried out in two ways. The first is based on throughput calculation. Throughput measures the amount of work done per unit of time. The formula used is:

$$\text{Throughput} = \frac{1}{T_{\text{exec}}} \cdot \frac{\sum_{t=1}^{K-1}(N - t(R-1))}{10^6} \text{ Mneurons/s}$$

Figure 3b shows the throughput results for different problem sizes. The throughput exhibited a sharp peak at a $N \approx 200000$, followed by a steep drop and a slower rise.



(a) Wall-clock times

(b) Throughput

Figure 3: Performance comparison between the OpenMP implementation and the CUDA implementation (global kernel)

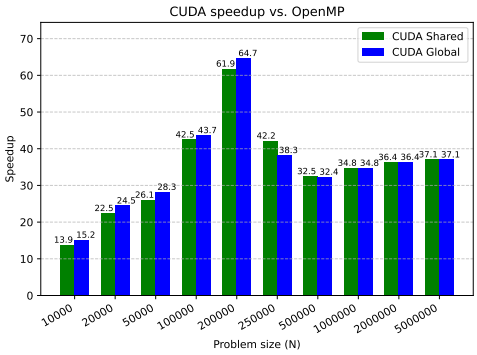


Figure 4: CUDA speedup relative to OpenMP

The second method compares the execution time of a CUDA program against a highly optimized, multi-threaded CPU implementation. As shown in Figure 3a, the execution time of the OpenMP version is consistently higher than that of the CUDA version.

Finally, we measured the speedup of CUDA relative to OpenMP. Figure 4 shows that the shared-memory CUDA version achieves a speedup of approximately 65× compared to the OpenMP implementation. Contrary to our expectations, however, the shared-memory kernel was consistently slower than the global-memory kernel across all tested problem sizes. To investigate this behavior, profiling was performed with Nsight Compute. The analysis revealed that the result was not due to a programming error, but rather to the limited amount of L2 cache available on the RTX 3090.

The shape of the throughput graph is a direct consequence of the RTX 3090's L2 cache:

- For $N < 200{,}000$, the problem's working set fits entirely within the fast L2 cache. The GPU can exploit the L2's high bandwidth, leading to a steep increase in throughput.
- For $N > 200{,}000$, the working set exceeds the L2 cache capacity. This results in frequent cache misses, forcing the GPU to fetch data from the much slower VRAM. Memory latency thus becomes the primary bottleneck, causing a sudden

performance degradation.

GPUs with larger L2 caches, such as enterprise models, are not expected to encounter this cliff. Moreover, profiling revealed that the shared-memory kernel achieved lower occupancy (84.06%) compared to the global kernel (88.15%). The shared kernel allocates static shared memory, which limits the number of thread blocks that can reside on an Streaming Multiprocessor (SM) simultaneously. Another insight came from comparing the L1/TEX Hit Rate of both kernels. The global kernel allows the hardware to use a large portion of the 128 KB on-chip memory per SM as an L1 cache. This resulted in an L1 Hit Rate of 78.09%. In the shared kernel, however, a portion of on-chip memory is reserved, reducing the available L1 cache. Consequently, caching became less effective for the remaining global memory accesses (primarily to the W matrix), causing the L1 Hit Rate to drop to 74.04%. The remaining 26% of accesses miss L1 and propagate to the L2, which is small. If the problem fits within L2, these accesses are still fast. As soon as N increases, L2 misses occur, forcing accesses to the much slower VRAM.

4 Conclusion

In conclusion, the CUDA implementation significantly outperforms the OpenMP implementation. OpenMP shows good scaling up to the number of Performance cores, after which efficiency decreases when hyper-threading is used. Efficient cores are slower and lack hyper-threading. CUDA achieves a speedup of up to approximately $65\times$ compared to OpenMP. The GPU can execute tens of thousands of threads, far exceeding the CPU's 10 cores. Moreover, the GPU's high-bandwidth memory provides significantly more bandwidth than the CPU's DDR4 memory, which is critical for this memory-bound application. Finally, we observed that the global-memory version performed better than the shared-memory version, likely due to the L2 cache limitations. This highlights how GPU optimizations are not always intuitive and depend strongly on the specific hardware (cache size, occupancy).